Design Facebook - a social network

Facebook is an online social networking service where users can connect with other users to post and read messages. Users access Facebook through their website interface or mobile apps.

Welcome to Facebook
https://www.facebook.com
facebook
Facebook helps you connect and share with the people in your life.

System Requirements

We will focus on the following set of requirements while designing Facebook:

1. Each member should be able to add information about their basic profile, work experience, education, etc.
2. Any user of our system should be able to search other members, groups or pages by their name.
3. Members should be able to send and accept/reject friend requests from other members.
4. Members should be able to follow other members without becoming their friend.
5. Members should be able to create groups and pages, as well as join already created groups, and follow pages.
6. Members should be able to create new posts to share with their friends.
7. Members should be able to add comments to posts, as well as like or share a post or comment.
8. Members should be able to create privacy lists containing their friends. Members can link any post with a privacy list to make the post visible only to the members of that list.
9. Any member should be able to send messages to other members.
10. Any member should be able to add a recommendation for any page.
11. The system should send a notification to a member whenever there is a new message or friend request or comment on their post.
12. Members should be able to search through posts for a word.

Extended Requirement: Write a function to find a connection suggestion for a member.

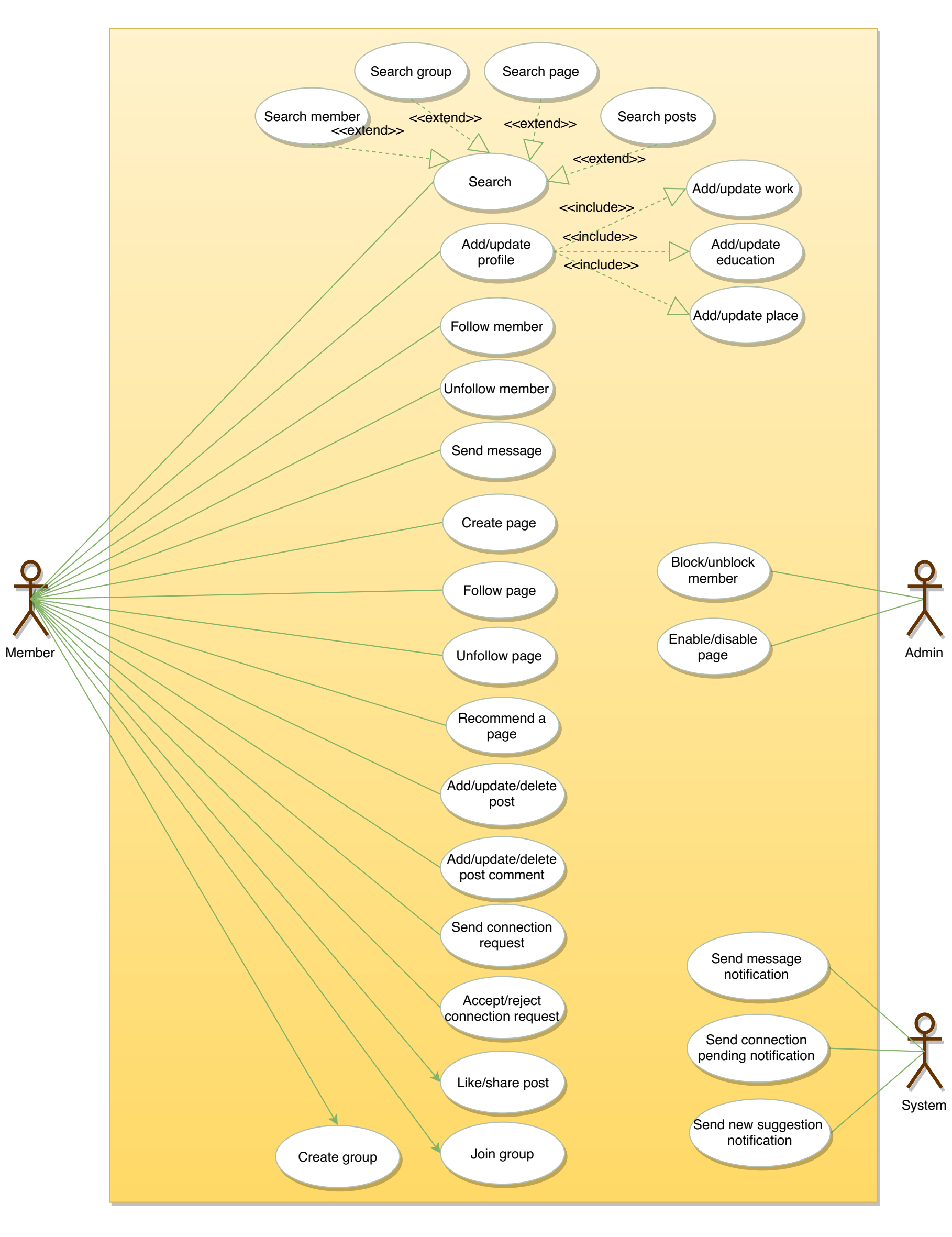
Use case diagram

We have three main Actors in our system:

- **Member**: All members can search for other members, groups, pages, or posts, as well as send friend requests, create posts, etc.
- **Admin**: Mainly responsible for admin functions like blocking and unblocking a member, etc.
- **System**: Mainly responsible for sending notifications for new messages, friend requests, etc.

Here are the top use cases of our system:

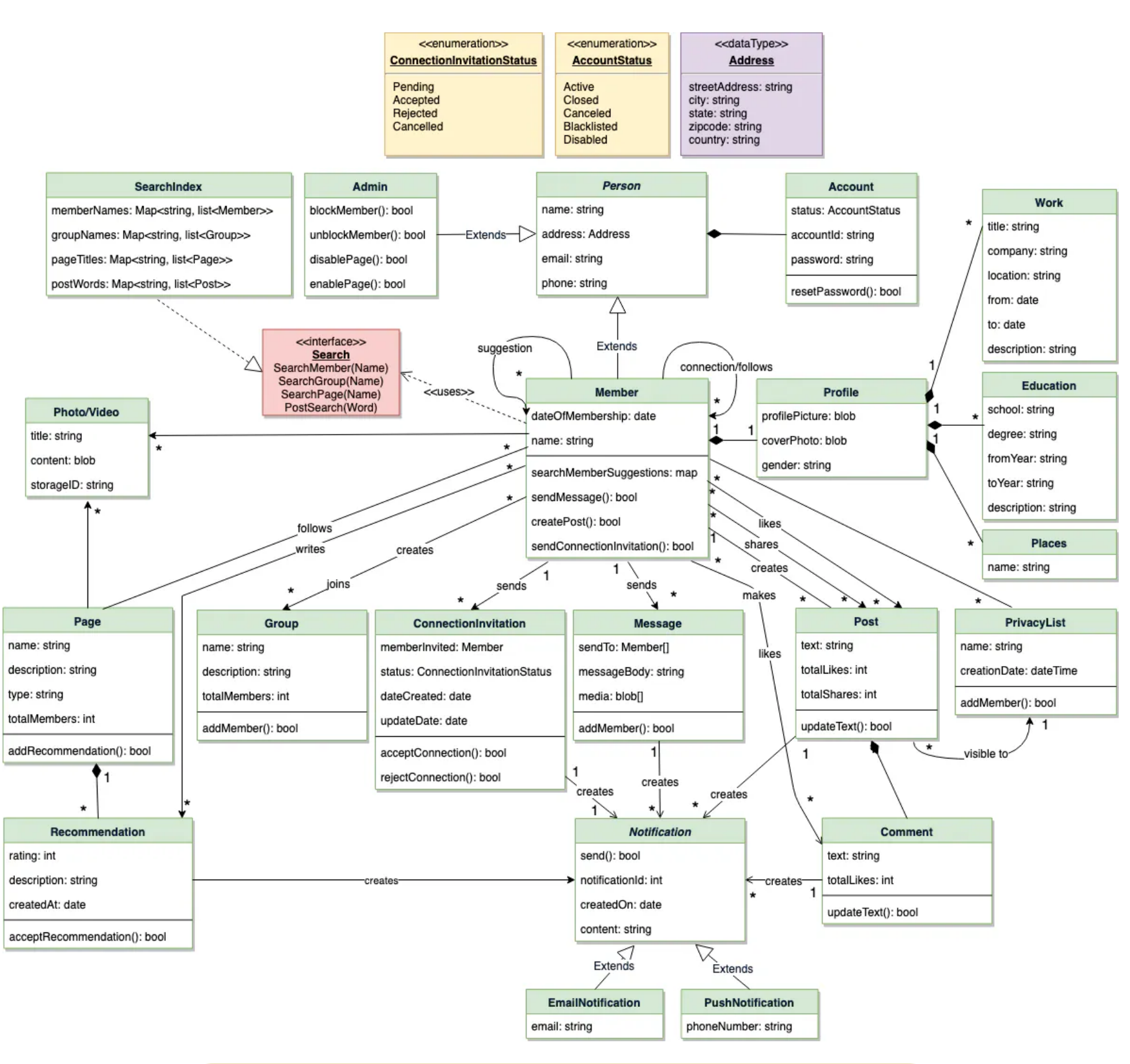
- **Add/update profile**: Any member should be able to create their profile to reflect their work experiences, education, etc.
- **Search**: Members can search for other members, groups or pages. Members can send a friend request to other members.
- **Follow or Unfollow a member or a page**: Any member can follow or unfollow any other member or page.
- **Send message** Any member can send a message to any of their friends.
- **Create post** Any member can create a post to share with their friends, as well as like or add comments to any post visible to them.
- **Send notification** The system will be able to send notifications for new messages, friend requests, etc.

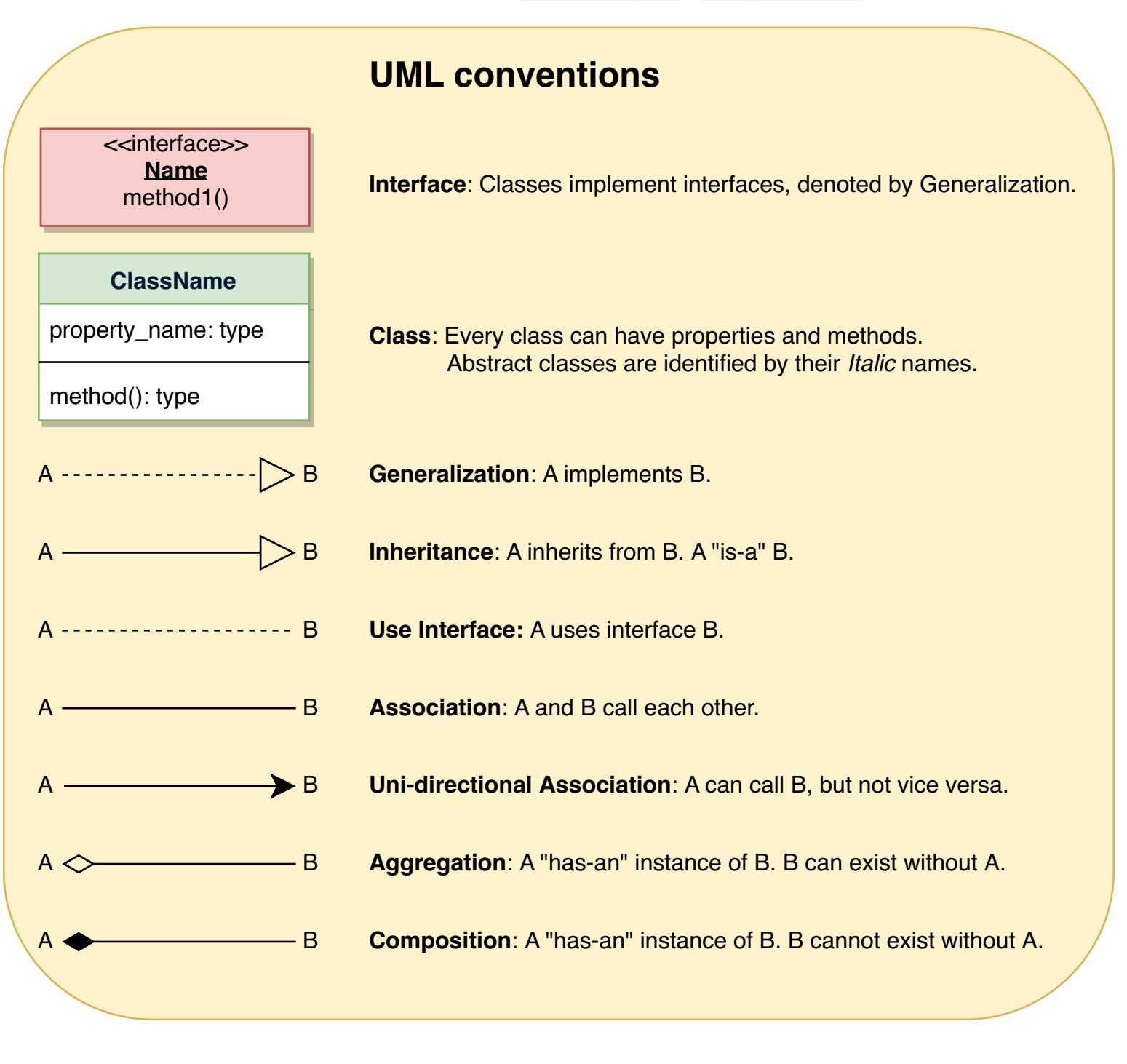

Search group
Search page
Search member
Search posts
<<extend>>
<<extend>>
<<extend>>
<<extend>>
Search
Add/update work
Add/update profile
<<include>>
<<include>>
<<include>>
Add/update education
Add/update place
Follow member
Unfollow member
Send message
Create page
Follow page
Unfollow page
Recommend a page
Add/update/delete post
Add/update/delete post comment
Send connection request
Accept/reject connection request
Like/share post
Join group
Create group
Member
Block/unblock member
Enable/disable page
Admin
Send message notification
Send connection pending notification
Send new suggestion notification
System

Class diagram

Here are the main classes of the Facebook system:

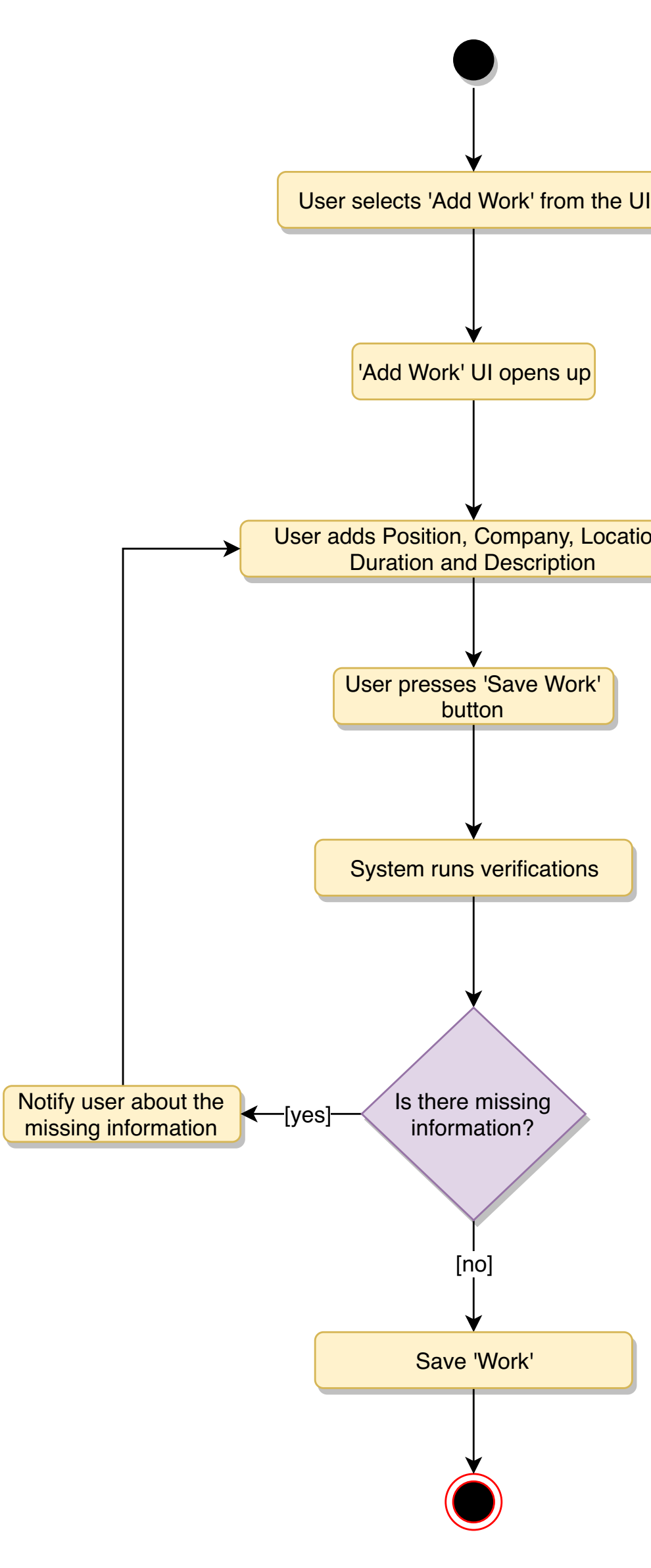
- **Member**: This will be the main component of our system. Each member will have a profile which includes their Work Experiences, Education, etc. Members will be connected to other members and they can follow other members and pages. Members will also have suggestions to send friend requests to other members.
- **Search**: Our system will support searching for other members, groups and pages by their names, and through posts for any word.
- **Message**: Members can send messages to other members with text, photos, and videos.
- **Post**: Members can create posts containing text and media, as well as like and share a post.
- **Comment**: Members can add comments to posts as well as like any comment.
- **Group**: Members can create and join groups.
- **PrivacyList**: Members can create privacy lists containing their friends. Members can link any post with a privacy list, to make the post visible only to the members of that list.
- **Page**: Members can create pages that other members can follow, and share messages there.
- **Notification**: This class will take care of sending notifications to members. The system will be able to send a push notification or an email.


<<enumeration>> ConnectionInvitationStatus
Pending
Accepted
Rejected
Cancelled
<<enumeration>> AccountStatus
Active
Closed
Canceled
Blacklisted
Disabled
<<dataType>> Address
streetAddress: string
city: string
state: string
zipcode: string
country: string
SearchIndex
memberNames: Map<string, list<Member>>
groupNames: Map<string, list<Group>>
pageTitles: Map<string, list<Page>>
postWords: Map<string, list<Post>>
Admin
blockMember(): bool
unblockMember(): bool
disablePage(): bool
enablePage(): bool
Extends
Person
name: string
address: Address
email: string
phone: string
Account
status: AccountStatus
accountId: string
password: string
resetPassword(): bool
Work
title: string
company: string
location: string
from: date
to: date
description: string
<<interface>> Search
SearchMember(Name)
SearchGroup(Name)
SearchPage(Name)
PostSearch(Word)
<<uses>>
suggestion
Extends
connection/follows
Member
dateOfMembership: date
name: string
searchMemberSuggestions: map
sendMessage(): bool
createPost(): bool
sendConnectionInvitation(): bool
Profile
profilePicture: blob
coverPhoto: blob
gender: string
Education
school: string
degree: string
fromYear: string
toYear: string
description: string
Places
name: string
Photo/Video
title: string
content: blob
storageID: string
follows
writes
creates
joins
sends
sends
likes
shares
creates
makes
likes
Page
name: string
description: string
type: string
totalMembers: int
addRecommendation(): bool
Group
name: string
description: string
totalMembers: int
addMember(): bool
ConnectionInvitation
memberInvited: Member
status: ConnectionInvitationStatus
dateCreated: date
updateDate: date
acceptConnection(): bool
rejectConnection(): bool
Message
sendTo: Member[]
messageBody: string
media: blob[]
addMember(): bool
Post
text: string
totalLikes: int
totalShares: int
updateText(): bool
PrivacyList
name: string
creationDate: dateTime
addMember(): bool
visible to
Recommendation
rating: int
description: string
createdAt: date
acceptRecommendation(): bool
creates
Notification
send(): bool
notificationId: int
createdOn: date
content: string
creates
Comment
text: string
totalLikes: int
updateText(): bool
Extends
Extends
EmailNotification
email: string
PushNotification
phoneNumber: string

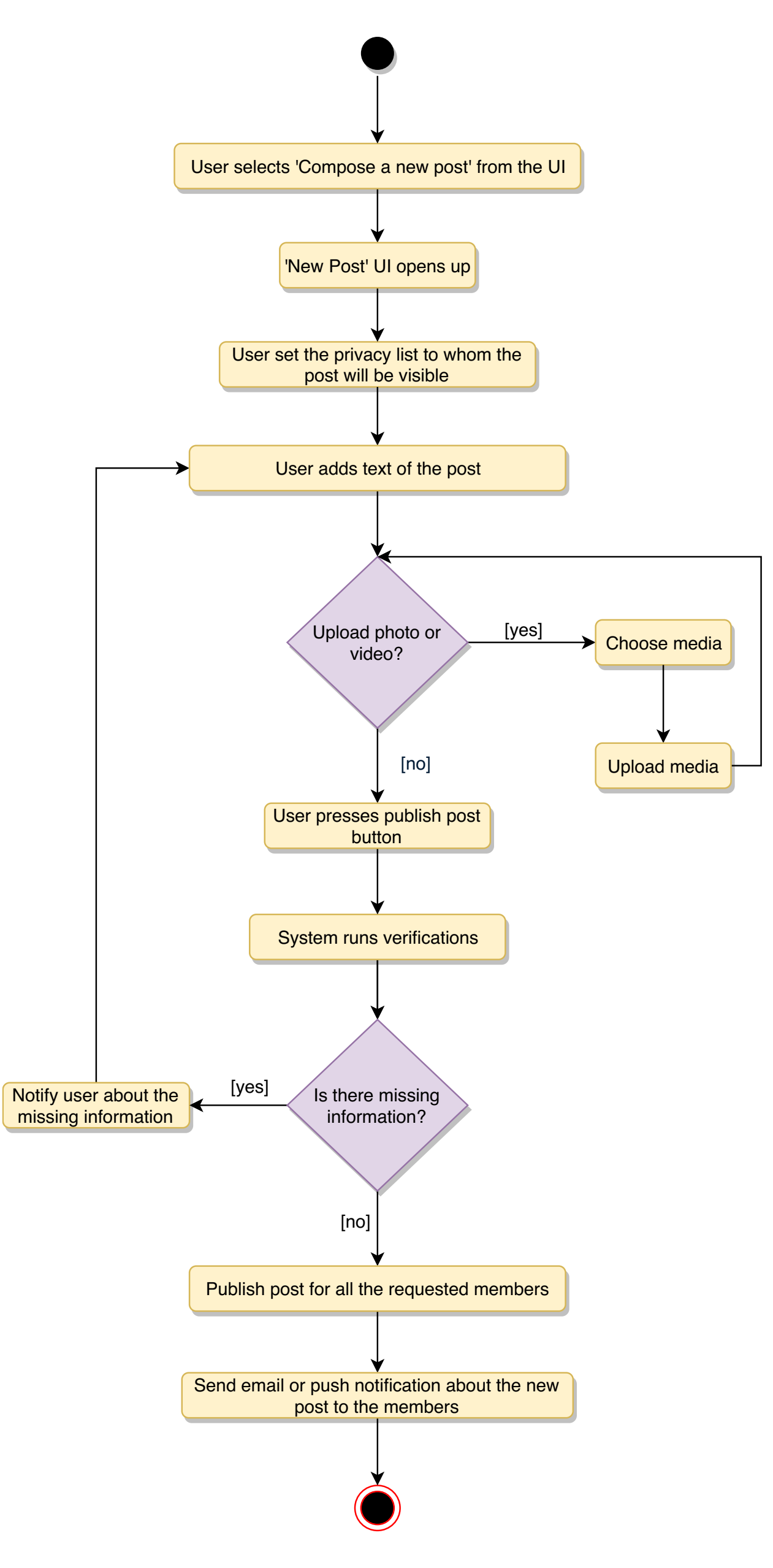

UML conventions
<<interface>> Name method1()
Interface: Classes implement interfaces, denoted by Generalization.
ClassName
property_name: type
method(): type
Class: Every class can have properties and methods. Abstract classes are identified by their Italic names.
A B Generalization: A implements B.
A B Inheritance: A inherits from B. A "is-a" B.
A B Use Interface: A uses interface B.
A B Association: A and B call each other.
A B Uni-directional Association: A can call B, but not vice versa.
A B Aggregation: A "has-an" instance of B. B can exist without A.
A B Composition: A "has-an" instance of B. B cannot exist without A.

Activity diagrams

Add work experience to profile: Any Facebook member can perform this activity. Here are the steps to add work experience to a member's profile:


User selects 'Add Work' from the UI
'Add Work' UI opens up
User adds Position, Company, Location, Duration and Description
User presses 'Save Work' button
System runs verifications
Is there missing information?
[yes]
Notify user about the missing information
[no]
Save 'Work'

Create a new post: Any Member can perform this activity. Here are the steps for creating a post:


User selects 'Compose a new post' from the UI
'New Post' UI opens up
User set the privacy list to whom the post will be visible
User adds text of the post
Upload photo or video?
[yes]
Choose media
Upload media
[no]
User presses publish post button
System runs verifications
Is there missing information?
[yes]
Notify user about the missing information
[no]
Publish post for all the requested members
Send email or push notification about the new post to the members

Code

Here is the high-level definition for the classes described above.

Enums, data types, and constants: Here are the required enums, data types, and constants:

Java

```java
public enum ConnectionInvitationStatus{
    PENDING,
    ACCEPTED,
    REJECTED,
    CANCELED
 }

 public enum AccountStatus{
    ACTIVE,
    CLOSED,
    CANCELED,
    BLACKLISTED,
    DISABLED
 }

 public class Address {
    private String streetAddress;
    private String city;
    private String state;
    private String zipCode;
    private String country;
```

Account, Person, Member, and Admin: These classes represent the different people that interact with our system:

Java

```java
// For simplicity, we are not defining getter and setter functions. The reader can
// assume that all class attributes are private and accessed through their respective
// public getter method and modified only through their public setter method.

public class Account {
  private String id;
  private String password;
  private AccountStatus status;

  public boolean resetPassword();
}

public abstract class Person {
  private String name;
  private Address address;
  private String email;
  private String phone;

  private Account account;
}

```

Profile and Work: A member's profile will have their work experiences, educations, places, etc:

Java

```java
public class Profile {
  private byte[] profilePicture;
  private byte[] coverPhoto;
  private String gender;

  private List<Work> workExperiences;
  private List<Education> educations;
  private List<Place> places;
  private List<Stat> stats;

  public boolean addWorkExperience(Work work);
  public boolean addEducation(Education education);
  public boolean addPlace(Place place);
}

public class Work {
  private String title;
  private String company;
  private String location;
  private Date from;
  private Date to;
```

Page and Recommendation: Each page can have multiple recommendations, and members will follow/like pages:

Java

```java
public class Page {
   private Integer pageId;
   private String name;
   private String description;
   private String type;
   private int totalMembers;
   private List<Recommendation> recommendation;

   private List<Recommendation> getRecommendation();
 }

 public class Recommendation {
   private Integer recommendationId;
   private int rating;
   private String description;
   private Date createdAt;
 }

```

Group, Post, Message, and Comment: Members can create posts, comment on posts, send messages and join groups:

Java

```java
public class Group {
   private Integer groupId;
   private String name;
   private String description;
   private int totalMembers;
   private List<Member> members;

   public boolean addMember(Member member);
   public boolean updateDescription(String description);
 }

 public class Post {
   private Integer postId;
   private String text;
   private int totalLikes;
   private int totalShares;
   private Member owner;
 }

 public class Message {
   private Integer messageId;
```

Search interface and SearchIndex: SearchIndex will implement Search to facilitate searching of members, groups, pages, and posts:

Java

```java
public interface Search {
    public List<Member> searchMember(String name);
    public List<Group> searchGroup(String name);
    public List<Page> searchPage(String name);
    public List<Post> searchPost(String word);
  }

  public class SearchIndex implements Search {
    HashMap<String, List<Member>> memberNames;
    HashMap<String, List<Group>> groupNames;
    HashMap<String, List<Page>> pageTitles;
    HashMap<String, List<Post>> posts;

    public boolean addMember(Member member) {
      if(memberNames.containsKey(member.getName())) {
        memberNames.get(member.getName()).add(member);
      } else {
        memberNames.put(member.getName(), member);
      }
    }

```

Extended requirement

Here is the code for finding connection suggestions for a member.

There can be many strategies to search for connection suggestions; we will do a two-level deep breadth-first search to find people who have the most connections with each other. These people could be good candidates for a connection suggestion, here is the sample Java code:

```java
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.stream.Collectors;
import static java.util.Collections.reverseOrder;

public class Member extends Person {
  private Integer memberId;
  private Date dateOfMembership;
  private String name;

  private Profile profile;
  private HashSet<Integer> memberFollows;
  private HashSet<Integer> memberConnections;
  private HashSet<Integer> pageFollows;
  private HashSet<Integer> memberSuggestions;
  private HashSet<ConnectionInvitation> connectionInvitations;
  private HashSet<Integer> groupFollows;

  public boolean sendMessage(Message message);
  public boolean createPost(Post post);
  public boolean sendConnectionInvitation(ConnectionInvitation invitation);

  private Map<Integer, Integer> searchMemberSuggestions() {
    Map<Integer, Integer> suggestions = new HashMap<>();
    for(Integer memberId : this.memberConnections) {
      HashSet<Integer> firstLevelConnections = new
Member(memberId).getMemberConnections());
      for(Integer firstLevelConnectionId : firstLevelConnections) {
        this.findMemberSuggestion(suggestions, firstLevelConnectionId);
        HashSet<Integer> secondLevelConnections = new
Member(firstLevelConnectionId).getMemberConnections());
        for(Integer secondLevelConnectionId : secondLevelConnections) {
          this.findMemberSuggestion(suggestions, secondLevelConnectionId);
        }
      }
    }

    // sort by value (increasing count), i.e., by highest number of mutual connection
count
    Map<Integer, Integer> result = new LinkedHashMap<>();
    suggestions.entrySet().stream()
        .sorted(reverseOrder(Map.Entry.comparingByValue()))
        .forEachOrdered(x -> result.put(x.getKey(), x.getValue()));

    return result;
  }

  private void findMemberSuggestion(Map<Integer, Integer> suggestions, Integer
connectionId) {
    // return if the proposed suggestion is already a connection or if there is a
    // pending connection invitation
    if(this.memberConnections.contains(connectionId) ||
        this.connectionInvitations.contains(connectionId)) {
      return;
    }

    int count = suggestions.containsKey(connectionId) ? suggestions.get(connectionId) :
0;
    suggestions.put(connectionId, count + 1);
  }
}
```